



TRƯỜNG ĐẠI HỌC THƯƠNG MẠI
Khoa HTTT Kinh tế và THMĐT
Bộ môn Công nghệ thông tin

Chương 2

QUY TRÌNH KIỂM THỬ PHẦN MỀM





NỘI DUNG

1. Giới thiệu
2. Các hoạt động kiểm thử
3. Sơ đồ tổ chức của đội kiểm thử
4. Ca kiểm thử, mô tả ca kiểm thử
5. Môi trường kiểm thử, dữ liệu kiểm thử
6. Thực hiện kiểm thử



Giới thiệu quy trình kiểm thử

- Làm rõ vai trò và trách nhiệm của việc kiểm thử phần mềm
- Làm rõ các công đoạn, các bước kiểm thử
- Làm rõ đầu vào, đầu ra mỗi bước kiểm thử



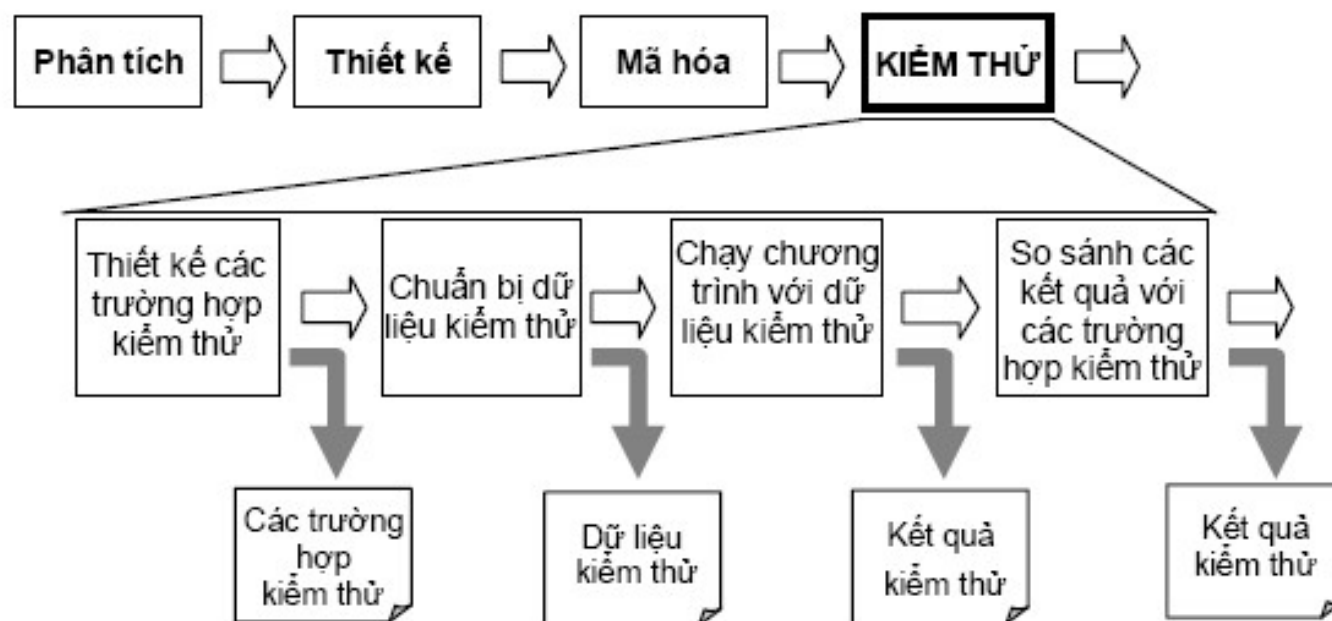
Các hoạt động kiểm thử





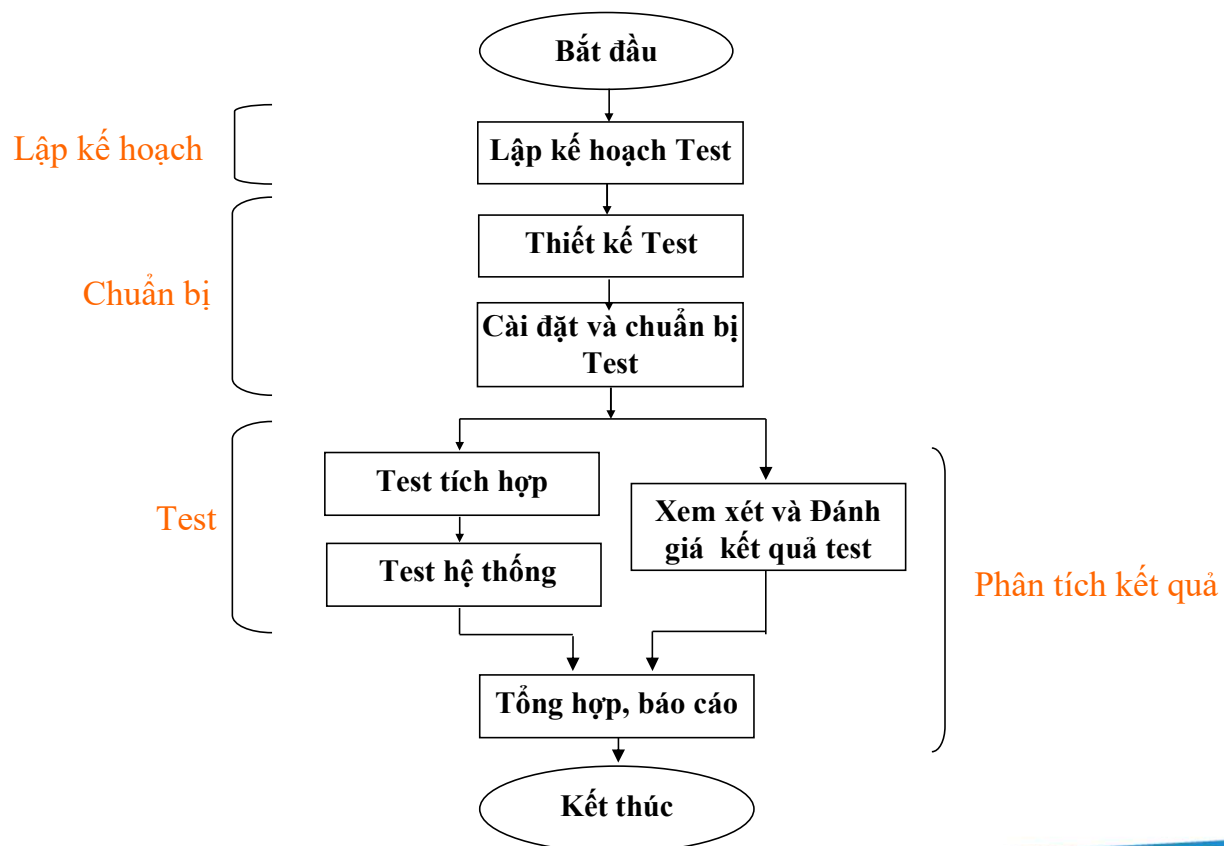
Quy trình kiểm thử phần mềm

- Quy trình kiểm thử phần mềm



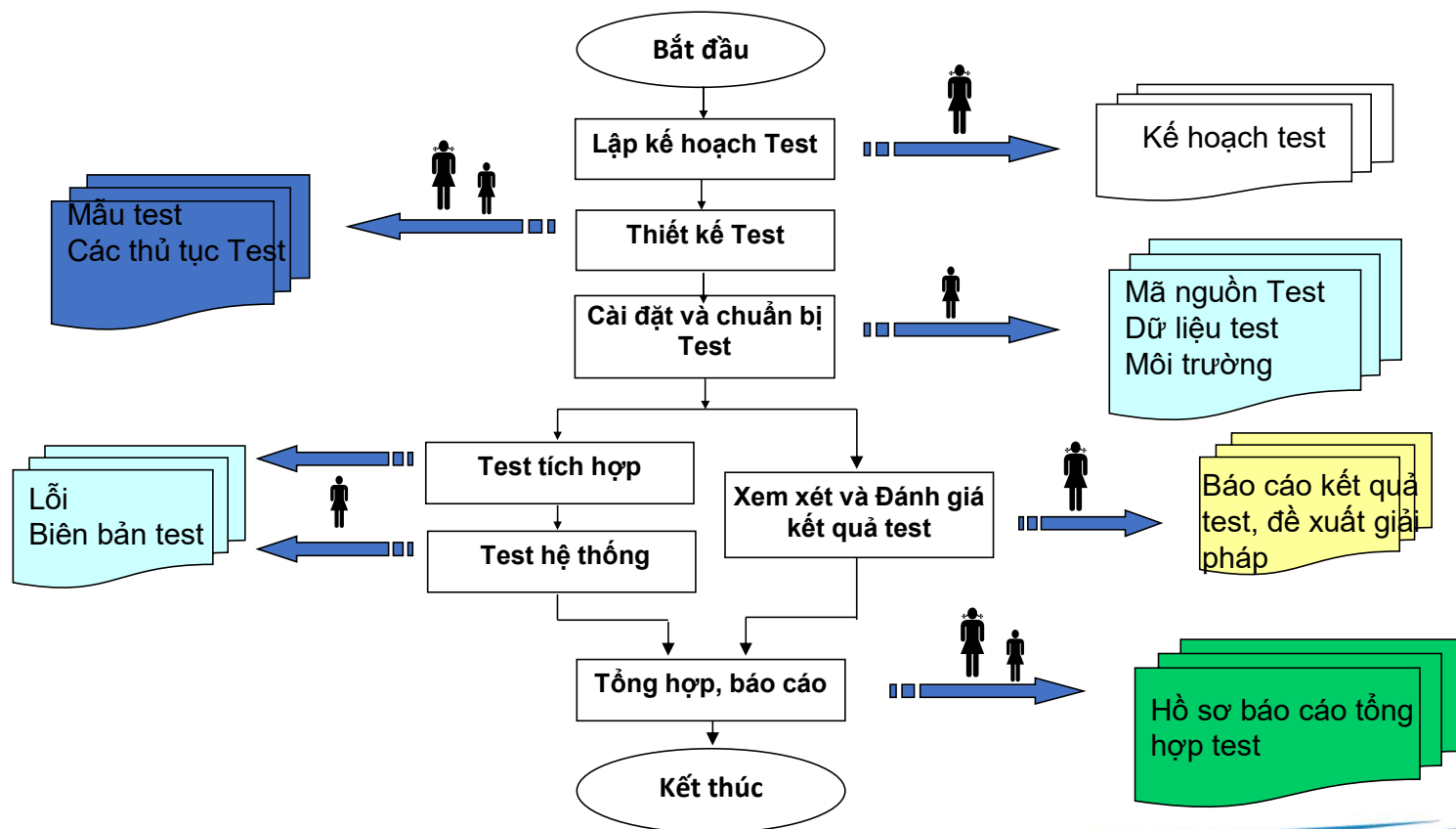


Quy trình kiểm thử phần mềm





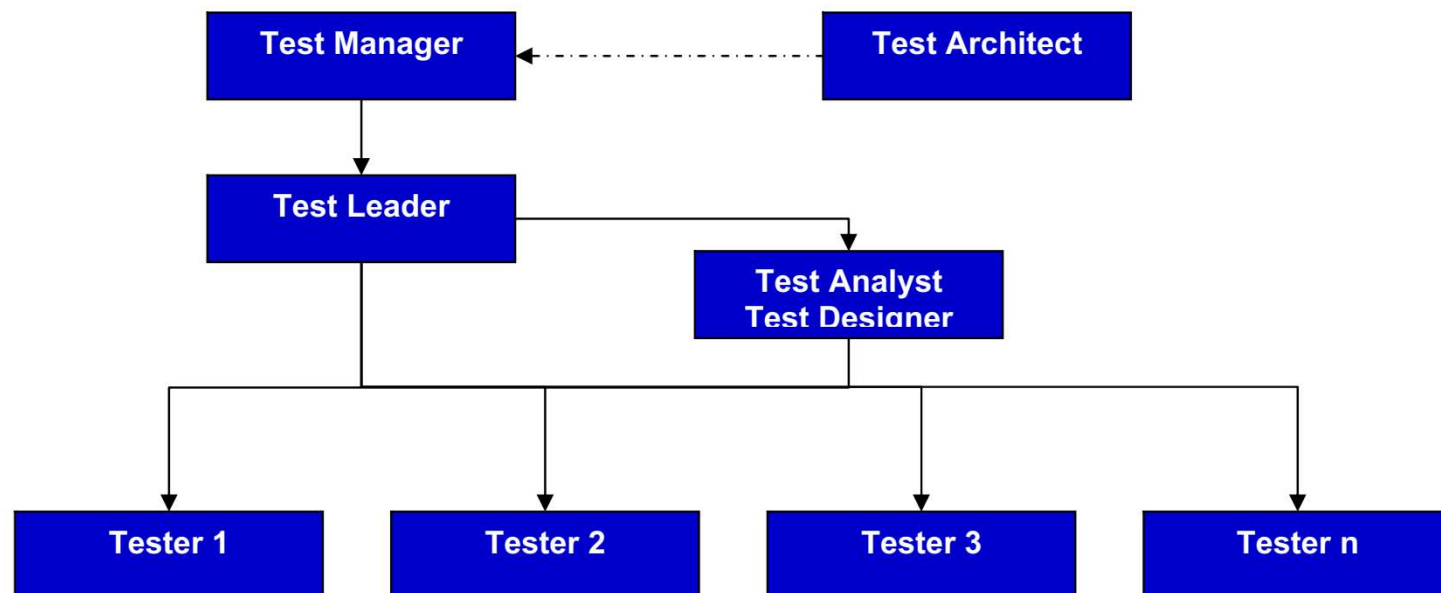
Quy trình kiểm thử phần mềm





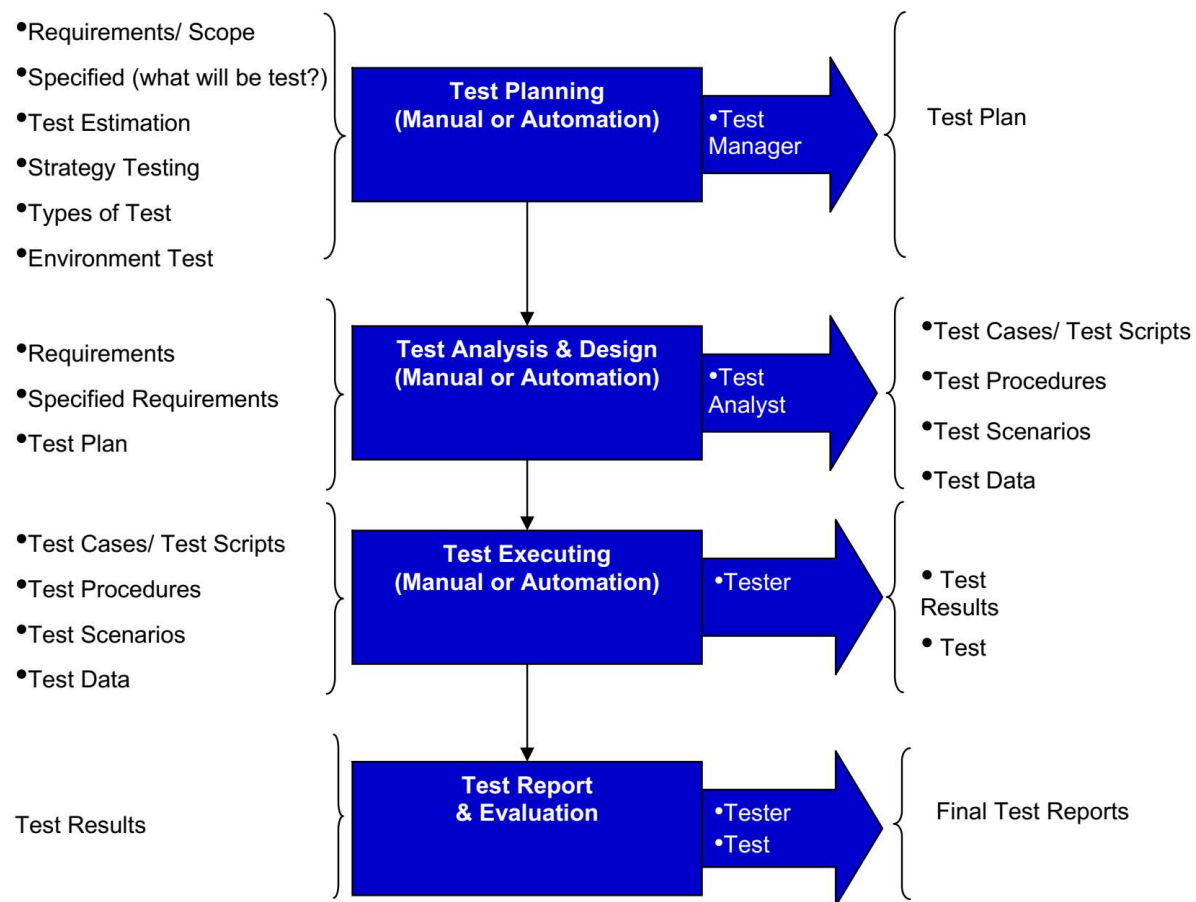
Sơ đồ tổ chức của đội kiểm thử

- Test manager
test leader
- Test analysis,
test designer
- Test worker
(tester)





Ai thực hiện mỗi bước kiểm thử





Lập kế hoạch kiểm thử

- Người thực hiện: test manager, test leader
- Xây dựng kế hoạch ban đầu cho hoạt động kiểm thử
 - Phạm vi kiểm thử
 - Gán trách nhiệm cụ thể cho từng thành viên nhóm kiểm thử
 - Các tài nguyên phần cứng, phần mềm, con người, môi trường kiểm thử
 - Xây dựng lịch trình kiểm thử
 - Nhận dạng các rủi ro và các yếu tố bất ngờ
 - Xác định hoạt động nào là thủ công, tự động hay cả hai
 - Ước lượng chi phí, tiêu chí dừng kiểm thử
- Hiệu chỉnh kế hoạch trong suốt chu kỳ kiểm thử để phản ánh các thay đổi nếu cần thiết



Lập kế hoạch kiểm thử

Xây dựng kế hoạch kiểm thử

- Định nghĩa phạm vi kiểm thử
- Định nghĩa chiến lược kiểm thử
- Nhận dạng các rủi ro và yếu tố bất ngờ
- Nhận dạng các hoạt động nào là thủ công, tự động
- Ước lượng chi phí và xây dựng lịch kiểm thử
- Nhận dạng môi trường kiểm thử

Kế hoạch kiểm thử cần được xem lại bởi Developers, Business Analysis, PM và Customers, cần được chấp thuận bởi PM, Customers, và được hiệu chỉnh trong suốt chu kỳ kiểm thử để phản ánh sự thay đổi nếu cần thiết.





Test Plan

- Test Plan là gì?
 - Là tài liệu chỉ ra test cái gì, theo phương pháp nào, khi nào, bởi a
- Tại sao phải làm Test plan?
 - Thảo luận: “plan” để làm gì?
- Ai làm Test plan?
 - Project Test Leader!
- Khi nào làm Test plan?
 - Càng sớm càng tốt, ngay từ khi nhóm dự án có Project Plan và Requirements Spec.
 - Quá muộn nếu làm Test plan khi cần bắt đầu thực hiện test.



Test Plan

- Nội dung căn bản của Test Plan
 - Giới thiệu tài liệu
 - Sơ lược hệ thống cần test và:
 - Các nhiệm vụ (mission) test trọng tâm
 - Các định hướng (motivator) để test
 - Các item cần test
 - Chiến lược test
 - Dùng phương pháp gì?
 - Có các testing stages nào?
 - Chọn những loại test nào?
 - Sử dụng những kỹ thuật test nào?
 - Sử dụng những công cụ nào?



Test Plan

- Nội dung căn bản của Test Plan
 - Các tài liệu, báo cáo test
 - Các vai trò, trách nhiệm của nhóm test
 - Tổ chức nhân sự và đào tạo
 - Tiến độ dự kiến



Thiết kế ca kiểm thử

- Người thực hiện: Test Analysis, Test Designer
- Thiết kế các ca kiểm thử dựa trên tài liệu đặc tả yêu cầu và kế hoạch kiểm thử
 - Thiết kế các ca kiểm thử cho yêu cầu chức năng và yêu cầu phi chức năng
 - Các ca kiểm thử cần bao phủ tất cả các khía cạnh kiểm thử cho từng yêu cầu
 - Nếu có kiểm thử tự động, cần xây dựng các kịch bản kiểm thử
 - Hiệu chỉnh, cập nhật khi có sự thay đổi



Thực hiện kiểm thử

- Người thực hiện: tester, customer
- Thực hiện/thi hành các ca kiểm thử
 - Thi hành kiểm thử theo từng ca kiểm thử
 - Kiểm thử lại các lỗi đã được sửa
 - Tạo báo cáo về lỗi trong suốt quá trình kiểm thử và theo dõi chúng đến khi chúng được xử lý
 - Riêng ở công đoạn kiểm thử chấp nhận, Customer (Khách hàng) sẽ thi hành kiểm thử để đánh giá phần mềm.



Báo cáo kiểm thử và đánh giá

- Người thực hiện: Test Manager, Test Leader
- Phân tích các lỗi trong hệ thống theo dõi các lỗi
- Đánh giá các kết quả kiểm thử, thống kê các yêu cầu thay đổi
- Tạo bảng tổng kết đánh giá hoạt động kiểm thử
- Xác định xem đã đạt tiêu chí thành công và hoàn thành kiểm thử chưa



CS4

- Tại sao phải làm Test plan?
 - Thảo luận: “plan” để làm gì?
- Tìm hiểu về 1 công cụ test tự động nào đó
 - Tên
 - Chức năng
 - Test cho giai đoạn nào



Tại sao phải có Test Plan?

1. Định hướng và tổ chức công việc: Test Plan cung cấp một cái nhìn tổng quan về mục tiêu kiểm thử, phạm vi, phương pháp và các hoạt động kiểm thử cần thực hiện. Nó giúp đội ngũ kiểm thử biết rõ mình phải làm gì, làm khi nào, và làm như thế nào.
2. Xác định phạm vi kiểm thử: Test Plan chỉ rõ những tính năng nào sẽ được kiểm thử và những tính năng nào sẽ không được kiểm thử, giúp tránh tình trạng kiểm thử lan man hoặc bỏ sót các tính năng quan trọng.
3. Quản lý nguồn lực: Test Plan giúp quản lý tốt hơn các nguồn lực, từ thời gian, nhân sự đến thiết bị, đảm bảo rằng mọi thứ đều được sắp xếp hợp lý và đúng theo kế hoạch.
4. Giảm rủi ro: Việc lập kế hoạch chi tiết cho các kịch bản kiểm thử và chiến lược kiểm thử trong Test Plan giúp phát hiện sớm các rủi ro có thể xảy ra trong quá trình kiểm thử, từ đó đề ra các biện pháp phòng ngừa phù hợp.
5. Đo lường và báo cáo tiến độ: Test Plan là cơ sở để đánh giá tiến độ kiểm thử và hiệu quả kiểm thử. Nó giúp theo dõi được công việc đã hoàn thành và những gì còn lại.
6. Phối hợp giữa các bên liên quan: Test Plan là tài liệu giao tiếp chính thức giữa các bên liên quan (quản lý dự án, đội phát triển, đội kiểm thử, khách hàng, v.v.), giúp mọi người có cùng hiểu biết về mục tiêu và tiến độ của quá trình kiểm thử.



Tại sao phải có Test Plan?

7. Tạo cơ sở cho việc ra quyết định: Test Plan giúp ban quản lý đưa ra các quyết định quan trọng dựa trên thông tin cụ thể và rõ ràng về chiến lược, tiến độ, và kết quả kiểm thử.
8. Đảm bảo chất lượng phần mềm: Với việc mô tả chi tiết về các phương pháp kiểm thử và các tiêu chuẩn chất lượng trong Test Plan, quá trình kiểm thử sẽ đảm bảo rằng phần mềm đáp ứng được các yêu cầu chất lượng đã đặt ra.
9. Quản lý thay đổi hiệu quả: Khi có các thay đổi về yêu cầu hoặc tính năng phần mềm, Test Plan có thể được cập nhật để phản ánh các thay đổi này, giúp đội ngũ kiểm thử điều chỉnh phương pháp kiểm thử một cách linh hoạt và phù hợp.
10. Hỗ trợ kiểm thử tự động hóa: Nếu kế hoạch kiểm thử bao gồm kiểm thử tự động, Test Plan sẽ định hướng việc chọn các công cụ tự động hóa, viết kịch bản tự động, và triển khai kiểm thử tự động hiệu quả, từ đó tiết kiệm thời gian và công sức.
11. Lưu trữ thông tin lịch sử: Test Plan là một tài liệu lịch sử quan trọng để lưu lại những kinh nghiệm và dữ liệu kiểm thử trong dự án, hỗ trợ quá trình phát triển và bảo trì phần mềm về sau.
12. Hỗ trợ cho kiểm thử phi chức năng: Bên cạnh kiểm thử chức năng, Test Plan cũng giúp lập kế hoạch cho các kiểm thử phi chức năng như kiểm thử hiệu suất, bảo mật, khả năng mở rộng, giúp đảm bảo phần mềm hoạt động tốt trong các điều kiện khác nhau.



Tại sao phải có Test Plan?

13. Tạo sự minh bạch và khả năng kiểm soát: Test Plan giúp tất cả các bên liên quan hiểu rõ quá trình kiểm thử, từ đó dễ dàng theo dõi, kiểm soát và điều chỉnh khi cần thiết. Điều này tạo ra sự minh bạch trong việc quản lý dự án, giúp loại bỏ hiểu nhầm và kỳ vọng không đúng đắn.
14. Đảm bảo tuân thủ quy trình và tiêu chuẩn: Trong các dự án phần mềm lớn hoặc những dự án yêu cầu tuân thủ các quy định và tiêu chuẩn công nghiệp (như ISO, CMMI), Test Plan sẽ giúp đảm bảo quy trình kiểm thử được thực hiện đúng theo các yêu cầu quy định.
15. Giảm thiểu xung đột: Test Plan giúp định nghĩa rõ ràng về trách nhiệm và vai trò của từng thành viên trong nhóm kiểm thử. Điều này giúp giảm thiểu xung đột hoặc hiểu nhầm giữa các bên tham gia, đảm bảo quy trình kiểm thử diễn ra một cách suôn sẻ.
16. Hỗ trợ việc đánh giá rủi ro kỹ thuật: Test Plan cung cấp cái nhìn tổng quan về các khu vực rủi ro kỹ thuật, giúp đội ngũ kiểm thử tập trung vào các tính năng hoặc phần mềm có khả năng gây lỗi cao nhất, từ đó giảm thiểu những vấn đề nghiêm trọng trong sản phẩm cuối cùng.
17. Tạo cơ sở cho các kịch bản kiểm thử chi tiết: Test Plan cung cấp nền tảng cho việc xây dựng các Test Case và Test Scenario chi tiết, giúp đảm bảo mọi trường hợp kiểm thử quan trọng đều được bao quát đầy đủ.
18. Giúp quá trình học hỏi và cải tiến: Thông qua việc lưu trữ thông tin và kết quả kiểm thử, Test Plan giúp đội ngũ học hỏi từ các sai sót hoặc những điểm cần cải thiện. Điều này hỗ trợ việc nâng cao quy trình kiểm thử trong tương lai, làm tăng hiệu quả và chất lượng phần mềm.
19. Công cụ đánh giá năng lực đội ngũ: Qua Test Plan, người quản lý có thể đánh giá được năng lực của đội ngũ kiểm thử, từ đó đưa ra các biện pháp cải thiện kỹ năng, đào tạo thêm hoặc điều chỉnh nhân sự phù hợp.



Ca kiểm thử - test case

Mô tả đơn giản:

- Một ca kiểm thử là một cặp đôi <đầu vào, đầu ra mong đợi>
- A test case = <input, expected output>

Mô tả chi tiết:

Định danh (tên) của ca kiểm thử

Mục đích

Tiền điều kiện

Đầu vào

Đầu ra mong đợi

Hậu điều kiện

Lịch sử thực hiện ca kiểm thử:

Ngày Kết quả thực tế Phiên bản Kiểm thử viên

Trình tự nhập dữ liệu/sự kiện vào giao diện người dùng?



Ví dụ về mô tả test case

1.3 Ca sử dụng tính học phí

❖ Mô tả ca sử dụng

Use case	Tính học phí
Tác nhân	Giảng viên
Mô tả	Giảng viên muốn tính học phí cho học viên
Điều kiện trước	Giảng viên đăng nhập thành công và được phân quyền người dùng
Điều kiện sau	Hệ thống hiển thị học phí của học viên
Luồng chính	1. Giảng viên nhập ID của học viên 2. Giảng viên nhập lớp học của học viên 3. Giảng viên nhập số buổi học của học viên 4. Giảng viên lưu dữ liệu đã nhập 5. Hệ thống xử lý dữ liệu 6. Hệ thống hiển thị học phí của học viên
Luồng phụ	- C1. Giảng viên nhập sai ID của học viên - C2. Giảng viên nhập sai lớp của học viên - C3. Giảng viên nhập sai số buổi học của học viên - C4. Giảng viên không nhập ID của học viên - C5. Giảng viên không nhập lớp của học viên - C6. Giảng viên không nhập số buổi học của học viên - C7. Giảng viên đột ngột thoát khỏi hệ thống



Ca kiểm thử

- Tập dữ liệu đầu vào (input): gồm các giá trị dữ liệu cần thiết để thành phần phần mềm dùng và xử lý
- Tập kết quả kỳ vọng (expected output): kết quả mong muốn sau khi thành phần phần mềm xử lý dữ liệu nhập
- Một phép thử được gọi là thành công nếu nó phát hiện ra khiếm khuyết của phần mềm. Kiểm thử chỉ chứng minh được sự tồn tại của lỗi trong hệ thống chứ không chứng minh được hệ thống không có lỗi
- Các ca kiểm thử nên được thiết kế khi chúng ta tạo các tài liệu phân tích và thiết kế, chứ không phải là khi đã viết xong mã nguồn.



Mô tả ca kiểm thử

- Các thành phần chính
 - Bộ giá trị đầu vào (input)
 - Đầu ra mong đợi (expected output)
 - Tạo ra gì: các giá trị được cung cấp bởi chương trình như con số, văn bản, âm thanh, ảnh, thông báo
 - Thay đổi gì (thay đổi giá trị của các biến, thay đổi CSDL)
 - Xóa gì
 - Thời gian
 - Trong điều kiện nào: file, net connection ...
 - Đầu ra thực tế (actual results)



Test Case Specification – đặc tả/mô tả ca kiểm thử

- Test case Spec là gì?
 - Là tài liệu trình bày về các test case sẽ được thực hiện.
- Ai làm Test case spec?
 - Tester
- Khi nào làm Test case spec?
 - Làm khi đã có test plan.
 - Trước khi bắt tay vào thực hiện test.



Test case spec

- Nội dung căn bản của Test case spec
 - Giới thiệu tài liệu
 - Phân nhóm và trình bày các test case. Thông tin về mỗi test case:
 - Mục đích thực hiện
 - Các bước thực hiện
 - Điều kiện trước khi thực hiện
 - Kết quả mong muốn
 - Dữ liệu test (nếu cần)



Module name

Reviewer:

	Module_name	Module_name1	Module_name2	Summary	
▶					+



CS5

1. Tìm hiểu về 1 công cụ test tự động nào đó

- Tên
- Chức năng
- Test cho giai đoạn nào

2. Xây dựng ca kiểm thử cho 1 chức năng của phần mềm/hệ thống bất kỳ



Ví dụ

- Kiểm thử tĩnh?
- Kiểm thử động
 - TC1=<[2,7,0], 1 >
 - TC2=<[0,7,2], 1 >

Ca kiểm thử nào có khả năng phát hiện lỗi/khiếm khuyết của chương trình?

```
/**
 * Counts zeroes in an array
 *
 * @param x array to count zeroes in
 * @return number of occurrences of 0 in x
 * @throws NullPointerException if x is null
 */
public static int numZero (int[] x)
{
    int count = 0;
    for (int i = 1; i < x.length; i++)
    {
        if (x[i] == 0) count++;
    }
    return count;
}
```



NHẮC LẠI VỀ KIỂM THỬ TĨNH VÀ ĐỘNG

- Code không được thực hiện (not execute). Nó có thể được thực hiện bằng tay hoặc bằng một bộ công cụ. Loại kiểm tra này thực hiện kiểm tra code, tài liệu yêu cầu và tài liệu thiết kế và đưa ra nhận xét, lưu nhận xét vào tài liệu công việc
- Được thực hiện trong môi trường thực thi chạy chương trình ứng dụng. Khi code được thực thi, thì đầu vào được truyền một giá trị, kết quả hoặc đầu ra của việc thực hiện được so sánh với kết quả dự kiến ban đầu đã đưa ra

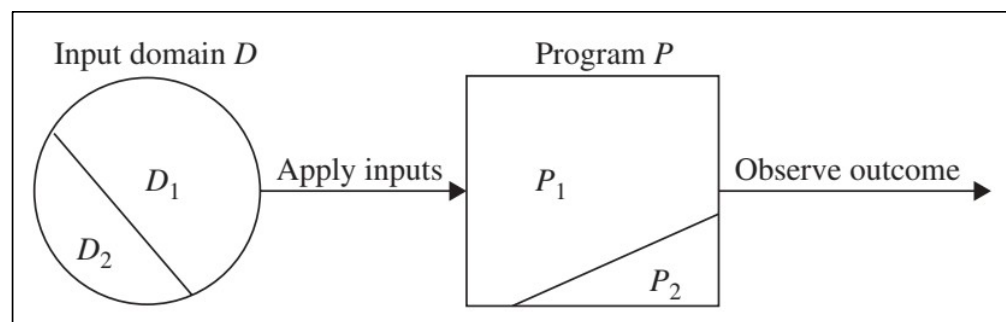


NHẮC LẠI VỀ KIỂM THỬ TĨNH VÀ ĐỘNG (2)

Static Testing	Dynamic Testing
1. Static testing chính là kiểm thử hộp trắng (white box testing) được thực hiện ở giai đoạn đầu của chu kỳ phát triển. Việc này hiệu quả hơn so với dung dynamic testing	1. Dynamic testing được thực hiện ở giai đoạn sau của vòng đời phát triển.
2. Trong thời gian ngắn hơn, thì Statis testing có nhiều phạm vi xác nhận hơn so với Dynamic testing	2. Dynamic testing có phạm vi xác nhận ít hơn vì nó chỉ bao gồm các khu vực giới hạn của code
3. Nó được thực hiện trước khi triển khai code	3. Nó được thực hiện sau khi triển khai code
4. Nó được thực hiện trong Bước Xác minh	4. Nó được thực hiện trong Giai đoạn Xác nhận
5. Loại thử nghiệm này được thực hiện mà không thực hiện code.	5. Loại thực hiện này được thực hiện với việc thực thi code.
6. Static testing đưa ra đánh giá của code cũng như tài liệu.	6. Dynamic testing cho biết giới hạn của hệ thống phần mềm.
7. Trong kỹ thuật Static testing một danh sách nội dung kiểm tra được chuẩn bị cho quá trình kiểm tra	7. Trong kỹ thuật Dynamic testing, các trường hợp thử nghiệm được thực thi.
8. Các phương pháp Static testing bao gồm Walkthroughs, code review.	8. Dynamic testing bao gồm việc kiểm tra chức năng và phi chức năng



Ca kiểm thử



- Chạy ca kiểm thử với chương trình P
 - Bao phủ một số yêu cầu của P ;
 - Bao phủ một phần chức năng P_1 (hoặc P_2) của P
 - Bao phủ một phần P_1 (hoặc P_2) trong cấu trúc của P
- => Tiêu chuẩn bao phủ sẽ định hướng thiết kế kiểm thử



Ca kiểm thử

- Mục tiêu thiết kế ca kiểm thử nhằm
 - *Tìm ra nhiều lỗi nhất*
 - *Với nỗ lực và thời gian nhỏ nhất.*
- Lựa chọn phương pháp kiểm thử nào đều phải đảm bảo:
 - *Tính đầy đủ (không sót một yêu cầu nào)*
 - *Cung cấp khả năng thật sự phát hiện được các lỗi.*
- Ca kiểm thử hiệu quả là ca kiểm thử phát hiện ra ít nhất 1 lỗi.



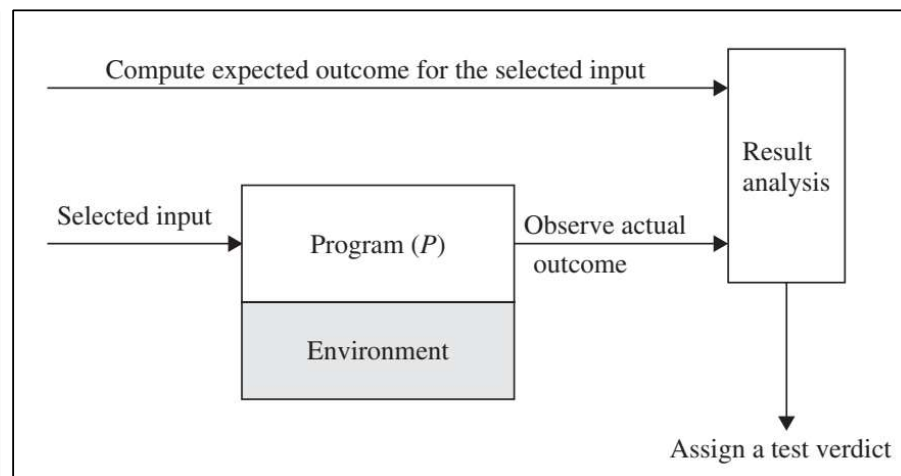
Ca kiểm thử tốt

- Ca kiểm thử thành công là ca kiểm thử trong đó hệ thống hoạt động sai
 - Lộ ra vấn đề của hệ thống
- Chọn ca kiểm thử (bộ dữ liệu đầu vào) có xác suất phát hiện lỗi/khiếm khuyết/thất bại cao hơn.



Các hoạt động kiểm thử

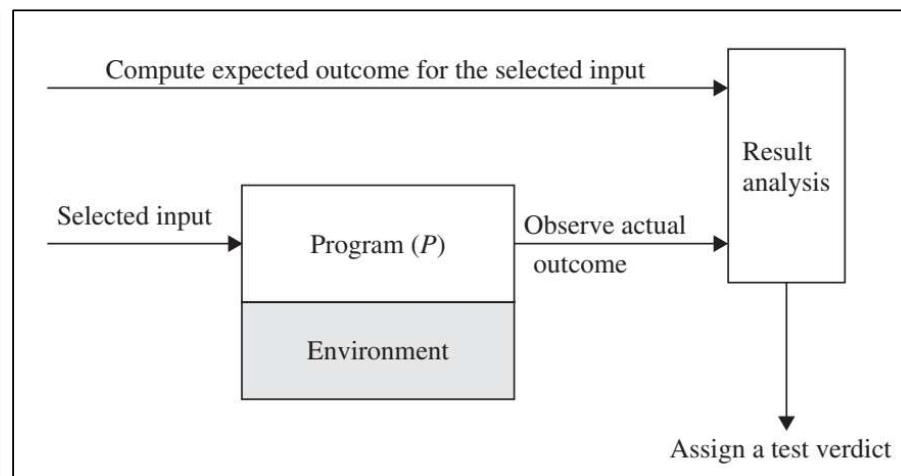
- Lựa chọn đầu vào cho ca kiểm thử: dựa vào tài liệu đặc tả yêu cầu, mã nguồn và mục tiêu kiểm thử
- Tính toán đầu ra mong đợi: dựa vào tài liệu đặc tả và mục tiêu kiểm thử
- Thiết lập môi trường thực hiện kiểm thử
 - Kết nối mạng Internet, cơ sở dữ liệu
 - Các hệ thống bên ngoài, từ xa trong hệ phân tán, máy khách hoặc máy chủ trong hệ thống Client-Server





Các hoạt động kiểm thử (tiếp)

- Thực hiện kiểm thử: thực thi chương trình với đầu vào đã chọn và quan sát kết quả hệ thống xuất ra
- Phân tích kết quả kiểm thử: so sánh kết quả thực tế với kết quả mong đợi rồi đưa ra kết luận về kết quả kiểm thử
- Viết báo cáo kiểm thử: cung cấp thông tin chi tiết về kết quả kiểm thử
 - Failure: giải thích cách tạo ra lỗi hoặc điều gì gây ra lỗi





Bài tập thiết kế ca kiểm thử

- ***Yêu cầu: kiểm thử chức năng đăng nhập hệ thống quản lý côngdaotao của Trường Đại học thương mại.***
- Link ứng dụng: <http://congdaotao.tmu.edu.vn/>
- Ứng dụng được kiểm thử trên 3 trình duyệt: Mozilla Firefox, Internet Explore, Googlechrome.



Bài tập thiết kế ca kiểm thử



Đăng nhập

Cổng thông tin đào tạo

Tên đăng nhập

1293

Tên đăng nhập là bắt buộc

Mật khẩu

Đăng nhập



Bài tập thiết kế ca kiểm thử

- **Phân tích:** Chức năng đăng nhập: yêu cầu kiểm thử chức năng đăng nhập thuần túy vào các ứng dụng web thông thường như yahoo, gmail. Các yếu tố cần kiểm tra:
 - Nếu *đăng nhập đúng tên và mật khẩu* thì tải đến trang chủ của ứng dụng.
 - Nếu *đăng nhập sai tên hoặc mật khẩu* thì đưa ra thông báo: “Tên đăng nhập hoặc mật khẩu đăng nhập không đúng”.
 - Nếu *nhập thiếu tên đăng nhập* thì đưa ra thông báo: “Bạn phải nhập tên đăng nhập”.
 - Nếu *nhập thiếu mật khẩu* thì đưa ra thông báo: “Bạn phải nhập mật khẩu”.



Bài tập thiết kế ca kiểm thử

- Lựa chọn phương pháp kiểm thử
 - Kiểm thử tĩnh (thủ công)
 - Kiểm thử trên máy



Bài tập thiết kế ca kiểm thử

- ***Xây dựng kịch bản kiểm thử tĩnh*** (thủ công): kịch bản kiểm thử chức năng Đăng nhập hệ thống thường được chia thành ba phần chính:
 - **Giao diện:** Kiểm thử các yếu tố giao diện chung như kiểm tra giao diện theo thiết kế, kiểm tra khi ấn tab, shift-tab, kiểm tra việc bị vỡ giao diện hay không, các giá trị mặc định của textbox.
 - Kiểm tra các item trên màn hình có đúng theo thiết kế chưa (***kiểm tra nội dung nhãn, màu sắc, link, status, canh lề***,....của từng item)
 - Kiểm tra validate cho các input item, mỗi item có nhiều vấn đề để kiểm tra như ***số ký tự tối thiểu, tối đa, cho phép nhập số, chữ hay ngày tháng, cho nhập ký tự đặc biệt không***.



Bài tập thiết kế ca kiểm thử

- **Chức năng:** Có bốn trường hợp chính cần kiểm thử:
 - Kiểm tra đăng nhập thành công với Tên đăng nhập/ Mật khẩu hợp lệ
 - Kiểm tra đăng nhập không thành công khi sử dụng sai Tên đăng nhập/ Mật khẩu
 - Kiểm tra thông báo khi không nhập Tên đăng nhập
 - Kiểm tra thông báo khi không nhập mật khẩu



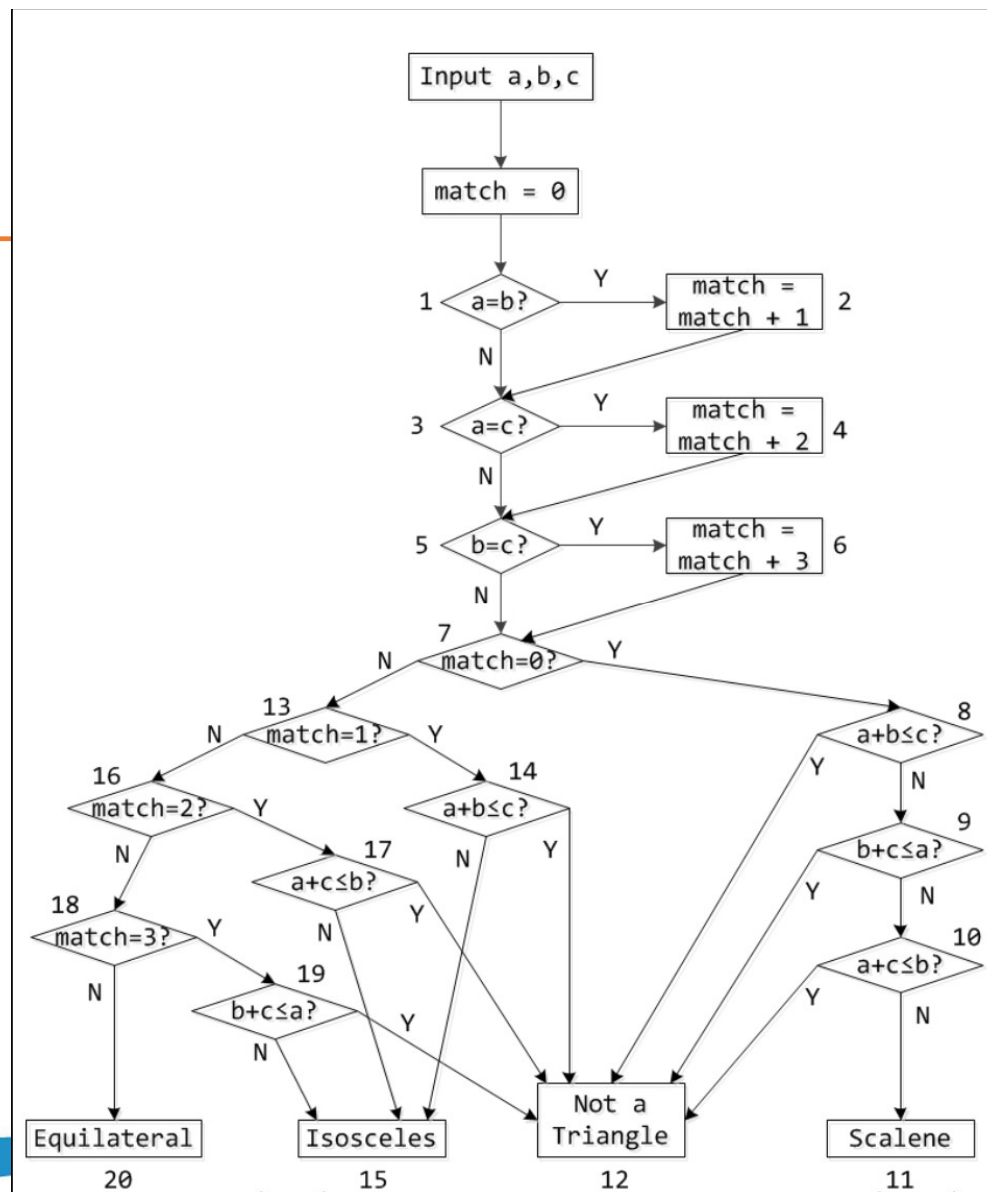
Bài toán tam giác

- Tính chất của tam giác: tổng của hai cạnh bất kỳ lớn hơn cạnh còn lại: $a < b + c$, $b < a + c$ và $c < a + b$
- Yêu cầu: viết chương trình nhập vào các cạnh a, b, c của tam giác và in ra màn hình thông báo phù hợp
 - Đây là tam giác thường
 - Đây là tam giác cân
 - Đây là tam giác đều
 - Đây không phải là tam giác



Bài toán tam giác

- Sơ đồ khối minh họa các bước thực hiện của chương trình





```
int a, b, c, IsATriangle;
```

```
//Function 1: Get Input
```

```
printf("Enter 3 sides (integers) of a triangle");
```

```
printf("a = ");
```

```
scanf("%d",&a);
```

```
printf("b = ");
```

```
scanf("%d",&b);
```

```
printf("c = ");
```

```
scanf("%d",&c);
```

```
printf ("Side A is %d\n", a);
```

```
printf ("Side B is %d\n", b);
```

```
printf ("Side C is %d\n", c);
```

```
//Function 2: Is A Triangle?
```

```
if((a < b + c) && (b < a + c) && (c < a + b))
```

```
    IsATriangle = 1;
```

```
else IsATriangle = 0;
```

```
//Function 3: Determine Triangle Type
```

```
if(IsATriangle)
```

```
    if((a == b) && (b == c))
```

```
        printf("Triangle is Equilateral");
```

```
    else if((a != b) && (a != c) && (b != c))
```

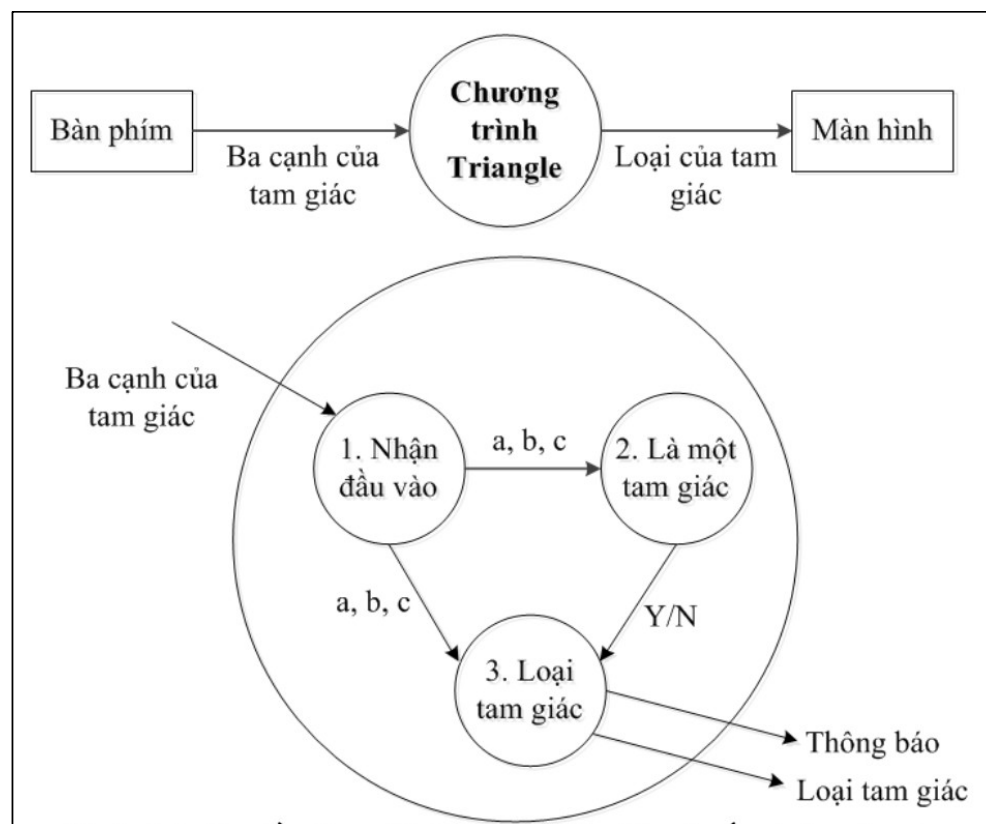
```
        printf("Triangle is Scalene");
```

```
    else printf("Triangle is Isosceles");
```

```
else printf("Not a Triangle");
```

```
return 0;
```

Bài toán tam giác





Bài toán tam giác

- Yêu cầu viết chương trình nhập vào độ dài các cạnh tam giác và in ra màn hình thông báo đó là ba cạnh của tam giác nào (tam giác thường, tam giác cân hay không phải là tam giác)
- Xác định tối thiểu 10 test cases và thực hiện kiểm thử chương trình đã viết
 - Ví dụ: [-3,-3,5]



Môi trường kiểm thử

- Phần mềm, phần cứng, cơ sở dữ liệu được sử dụng để thực hiện kiểm thử
- Phần mềm cần kiểm tra
- Máy chủ, máy khách
- Hệ điều hành, hệ quản trị cơ sở dữ liệu, trình duyệt
- Mạng
- Cơ sở dữ liệu (dữ liệu kiểm thử)
- Tài liệu hướng dẫn cài đặt, cấu hình, sử dụng



Dữ liệu kiểm thử

- Dữ liệu thực tế thường có yêu cầu được bảo mật: dữ liệu giao dịch ngân hàng, dữ liệu hồ sơ y tế, dữ liệu cơ quan chính phủ, ...
- Cần tạo hoặc thu thập dữ liệu phục vụ riêng cho hoạt động kiểm thử
 - Tạo dữ liệu thủ công: nhập từng bản ghi dữ liệu
 - Tạo dữ liệu tự động: nhập dữ liệu từ các tệp dữ liệu sẵn có hoặc sử dụng công cụ đọc, sinh dữ liệu tự động
 - Sao chép tự động từ dữ liệu của khách hàng



Dữ liệu kiểm thử

- Hai loại dữ liệu kiểm thử chính
 - Đầu vào của ca kiểm thử
 - Các bản ghi được lưu sẵn trong cơ sở dữ liệu trước khi thực hiện các ca kiểm thử
- Các loại dữ liệu
 - Dữ liệu “giả”: dữ liệu được cố tình làm sai lệch để thử nghiệm phản hồi của phần mềm
 - Dữ liệu chuẩn: xem phản hồi của phần mềm trong trường hợp dữ liệu chuẩn
 - Dữ liệu trống, dữ liệu mặc định



Thực hiện kiểm thử

- Thực hiện kiểm thử thủ công
- Thực hiện kiểm thử tự động



Kiểm thử thủ công

- Tester làm mọi công việc hoàn toàn bằng tay
 - Xác định test cases
 - Viết test cases
 - Chuẩn bị môi trường và dữ liệu kiểm thử
 - Thực hiện test cases: nhập điều kiện đầu vào, thực hiện một số sự kiện khác như click nút và quan sát kết quả thực tế, sau đó so sánh kết quả thực tế với kết quả mong muốn trong test case, điền kết quả test.
- Hiện nay, phần lớn các tổ chức, các công ty phần mềm, hoặc các nhóm làm phần mềm đều thực hiện kiểm thử thủ công là chủ yếu.



Kiểm thử tự động

- Thực hiện kiểm thử phần mềm bằng một chương trình đặc biệt với rất ít hoặc không có sự tương tác của con người
 - Dữ liệu kiểm thử được nhập từ các tệp sẵn có: file excel, ...
 - Thực hiện theo kịch bản kiểm thử được đọc từ tệp sẵn có
 - So sánh kết quả thực tế và kết quả mong đợi từ tệp sẵn có
 - Xuất báo cáo kiểm thử
- Giúp cho người thực hiện việc kiểm thử phần mềm (tester) không phải lặp đi lặp lại các bước nhàm chán
- Sinh các ca kiểm, dữ liệu thử từ tài liệu yêu cầu, tài liệu thiết kế, tài liệu chương trình



Bài tập chương 1,2

- Lựa chọn trong số các chương trình đã viết, thực hiện đầy đủ các bước trong quy trình kiểm thử
 - Xác định các ca kiểm thử
 - Mô tả các ca kiểm thử
 - Chuẩn bị môi trường, dữ liệu kiểm thử
 - Thực hiện kiểm thử
 - Viết báo cáo kết quả kiểm thử



BÀI TẬP

- 1 – Viết Test case cho chức năng đăng nhập của congdaotao.tmu.edu.vn
- 2- Tìm hiểu bài toán tam giác trong slides chương 2 => viết 10 test case yêu cầu.

Làm thành file word, nộp tại địa chỉ sau:

<https://forms.gle/jeqoShBZpUBnFaNr9>



CS6

- Dựa trên bài tập nhóm, thực hiện:

I. PHÁT BIỂU BÀI TOÁN	18
1. Mô tả bài toán	18
a. Quản lý sản phẩm:	18
b. Quản lý hóa đơn:	20
c. Quản lý kho:	21